Quality Code 0.44

The commands, and what is new. From Jira ticket to tests you can trust.

What is new since 0.28

- **Frontend unit tests, per feature (0.41).** A feature is tested end to end, with unit tests, or both, and can point to your team's own guide for it. Setup advanced asks; spec reads it before it plans.
- **The whole feature, not only the branch (0.41).** When a pull or merge request goes into another feature branch, spec asks which base to use: this branch against that one, or the whole feature against the main line. Merge request files are read from GitLab.
- **Files and the audit log stay in the session (0.41).** The last question of setup is where to keep them: the repo, this machine, or only the session.
- **Pilot-report fixes (0.42, 0.43).** Setup picks the project by what your branch changed and asks only when unsure; spec can write the tests first, before the code exists; the expected-value check reads tables and variables your tests feed to assertions; `externalChecks` counts database tests (pgTAP) for scenarios a unit test cannot check.
- **Traceability on this branch (0.44).** The KPI shows the share of all specs that name a ticket, and the share of the specs this branch changed.
- **QA flow (0.39).** Blocked is not failed, evidence per step, resume a run, retest with a linked defect, and Xray tests created through the API (tried with mocks only).
- **Local seed data (0.31).** seed now loads about 25 realistic rows per table into your local database, never a shared one.
- **CI with two jobs (0.33).** ci writes the file: a quick quality-gate and a quality-tests job for each pull or merge request.
- **Central Context rules (0.34, 0.40).** Spec reads the company's testing rules, your own local rules, and five draft rules shipped with the plugin.
- **Easier to read (0.35 to 0.38).** Progress as a live checklist, tables and colour marks, a report style, and a short wrap-up in plain words.
- **Other agents (0.32).** `quality-code-mcp --init copilot|codex|cursor|gemini` writes the config and wrappers for them.
- **check-update (0.29, 0.30).** Says whether a newer version is out, and updates it on your yes.

Update first

In Claude Code (CLI or the desktop Code tab):

```
/plugin update quality-code@teamsystem
/reload-plugins
```

Type /quality-code: and check that the list shows check-update, run-pipeline, spec, ci and routine. Or run /quality-code:check-update.

1. Set up (once)

Command	What it does	Example
/quality-code:setup	Checks the connectors (GitLab or GitHub, Jira, Figma), finds the project, its stack, the Jira project and the base branch, shows one card to confirm, and writes .claude/quality-code.json. The last question is where files and logs go. Option 2 is a personal setup for QA and PO with no repo.	`/quality-code:setup`
/quality-code:setup advanced	Feature labels, QA plans, title prefixes, templates, redaction, and the test approach per feature (end to end, unit, both).	`/quality-code:setup advanced`

Command	What it does	Example
/quality-code:config	Shows and changes one setting at a time: the project config, where files and logs go, your plugin settings (Jira email, GitLab, Xray), the reply style, and which tokens are set. Every change is checked before it is saved.	`/quality-code:config`
/quality-code:check-update	Says whether a newer version is out and what changed. Offers to update, then asks you to type /reload-plugins. Read-only until you say yes.	`/quality-code:check-update`

2. Every day: from ticket to tests

Command	What it does	Example
/quality-code:spec	The main command. Takes the ticket of your branch (or a key), runs the readiness gate, shows the plan, writes the tests in your test folders, runs them through the pipeline and reads the card back. It asks three times at most. When the request goes into a feature branch, it asks which base to use.	`/quality-code:spec` `/quality-code:spec ABC-123`
/quality-code:spec KEY --gate-only	Only the readiness score of a ticket, no tests.	`/quality-code:spec ABC-123 --gate-only`
/quality-code:spec KEY --base BRANCH	Compare against the branch you name, without asking.	`/quality-code:spec ABC-123 --base develop`
/quality-code:start, :write	Old names, kept: the same as spec (start = no key, write = with a key).	`/quality-code:write ABC-123`
/quality-code:ticket-eval	Scores one ticket out of 100 (70 passes). Refused: fill it in here, post a comment on Jira, or continue anyway (marked RISK).	`/quality-code:ticket-eval ABC-123`
/quality-code:ticket	Finds a ticket and shows it in short, with tips for its author. Takes a key or filters. Read-only.	`/quality-code:ticket assignee:me status:"In Progress"`

3. Run and grade the tests

Command	What it runs	Example
/quality-code:run-frontend	Every Playwright spec of the project.	`/quality-code:run-frontend`
/quality-code:run-backend	Every backend spec: Vitest, pytest, JUnit, Go or .NET, whichever the project uses.	`/quality-code:run-backend`
/quality-code:run-pipeline	Both together, the full suite. --branch: only the tests your branch changed. Spec paths: only those. --evals: the frozen evals.	`/quality-code:run-pipeline --branch`

All three take `--key KEY` (judge one ticket's tests) and `--flaky N` (run N times, catch flaky tests). The card ends in one verdict, decided by a script, never by the model. A red run now gives one fix per failing check and lists the failing tests apart.

Verdict	Meaning
green	The tests run, assert the ticket's values and catch changes to them.
tests red	The checks pass and some tests fail: a real bug, or a fix not merged yet.
checks failed	A problem with the tests themselves: hollow, a missing ticket value, a removed assertion.
not verified	Something could not run (the app is down, no type-check, no runner). Never counted as a pass.

4. CI and scheduled checks

Command	What it does	Example

/quality-code:ci	Writes the CI file for GitHub Actions or GitLab CI, with two jobs for each pull or merge request. **quality-gate** (seconds) checks the changed specs for honesty, assertions and the frozen evals. **quality-tests** installs the project and runs types, tests and mutation on the changed specs. A code-only change runs its side's whole suite without mutation. A file it wrote earlier is replaced, so run it again after an update.	`/quality-code:ci`
		`/quality-code:ci gitlab`
		`/quality-code:ci github --print`

/quality-code:routine run	Creates a schedule with Claude's scheduler: once or on a period. Fetches the latest commits, runs a task (--task KEY) or the whole suite, and reports a change from green to red with the commits in between.	`/quality-code:routine run --task ABC-123 --since-last`
/quality-code:routine status	Lists the routines, the last result and the history; stops a routine.	`/quality-code:routine status`
--stability N	Runs every test N times and gives a verdict per test: STABLE, UNSTABLE or NOT ENOUGH DATA, with the pass rate over time. Alias --flaky.	`/quality-code:routine run --stability 5`

5. For QA and product owners

Command	What it does	Example
/quality-code:tickets-eval	Scores a whole Jira project, read-only: one line per ticket, how many pass, what is missing most.	`/quality-code:tickets-eval ABC` `--user me`
/quality-code:xray	Writes Xray test cases from a ticket: a CSV and a case.xlsx per ticket to import into Xray. With the Xray API key set, it creates the tests in Xray too.	`/quality-code:xray ABC-123`
/quality-code:qa	Runs your Xray manual tests with an agent, test by test in the browser, and drafts each defect. You give every verdict; nothing reaches Jira without a preview. A test that cannot be done is Blocked, not failed. Answers are saved after each test: --resume goes on from them.	`/quality-code:qa` `/quality-code:qa --resume`
/quality-code:seed	Loads realistic mock data for a ticket into your local database (about 25 rows per table plus the ticket's own case), so you can test it by hand. Resets the rows it fills first. Local databases only.	`/quality-code:seed ABC-123`

6. Numbers

Command	What it shows	Example
/quality-code:kpi	Tickets gated, fill-ins, Jira comments, pipeline verdicts, and the Quality Program numbers: traceability (whole suite and this branch), completeness, suite honesty, gate integrity, coverage and mutation kill rate. Read-only.	`/quality-code:kpi --since 30d`
/quality-code:consumption	AI usage and estimated cost of the plugin's runs, read locally.	`/quality-code:consumption --by command`

Where files and logs go

By default they stay in the session, in a temporary folder. Setup asks, as its last question, whether to keep them in the project repo or on this machine instead. `/quality-code:config` changes it later. A script or CI job can set the default with `QUALITY_CODE_PLACE=repo|local|chat`.

Settings you may add to .claude/quality-code.json

Setting	What it does
features[].approach	How a feature is tested: e2e (default), unit or both. Unit tests need a JavaScript unit runner.
features[].guide	A file in the project that says how your team tests that feature. Spec reads it first, and it wins over the plugin's guide.
externalChecks	Files of tests that run outside the unit tests, for example a pgTAP file for a seeded table. Read, not run, by Quality Code.
git.base	The main line feature branches start from. Needed for the branch numbers and for the whole-feature check.
qa	Your Xray issue types and status names, and how many blocked tests in a row offer to stop a run.

MCP server (every command as a tool)

Mode	How	Notes
Local (stdio)	quality-code-mcp --transport stdio	No sign-in. Install with install.sh, check with --check, --print-config.
Remote (HTTP)	quality-code-mcp --transport http	Entra ID sign-in; only ticket, ticket-eval and tickets-eval; Docker image; open /health.
Other agents	quality-code-mcp --init copilot|codex|cursor|gemini	Shows what it would write; add --write to write it. Local only. Tried as generated files, not yet inside the four tools.

Remote needs the Entra tenant and client IDs, HTTPS, and a project folder with .claude/quality-code.json.

Backends

Stack	Runs with	Write tests like this
TypeScript, Node	Vitest	describe('ABC-123 ...') with one it() per scenario
Python	pytest	class Test... whose docstring starts with ABC-123; plain assert
Java, Kotlin	JUnit 5 or 4 (Maven, Gradle)	@DisplayName("ABC-123 ...") on the class
Go	go test	// ABC-123 ... above the test function, one t.Run per scenario
C#	dotnet test (xUnit, NUnit, MSTest)	// ABC-123 ... above the class
Frontend	Playwright	test.describe('ABC-123 ...', { tag: '@ABC-123' })

spec writes them this way for you; the rule matters only when you write tests by hand. Stryker is Vitest only. Jest, Mocha, node --test and Cypress run with the generic checks.

Old command, new command

Before	Now
/quality-code:check	/quality-code:run-pipeline --branch (or run-frontend / run-backend)
/quality-code:check spec.ts	/quality-code:run-pipeline spec.ts
/quality-code:check --evals	/quality-code:run-pipeline --evals
/quality-code:start, /quality-code:write KEY	still work; the same as /quality-code:spec
/quality-code:ci --all, --changed, --init github --write	/quality-code:ci github (the generated file does the choosing); --print shows it without writing
/quality-code:seed (test data printed)	/quality-code:seed loads mock data into the local database